Brian Walker

002615591

Bwalker76

Explain why you chose to use StatefulWidget for your calculator app. What specific pieces of data needed to be managed as state? How did you decide which variables should be part of the widget's state versus local variables in methods? Could you have implemented this app using StatelessWidget? Why or why not?

I chose a stateful widget because the calculator constantly has to update the UI for the user. Using a stateless widget wouldn't allow the calculator to work correctly. I decided which ones should go into the state based on what was being shown and what needed to change with the UI. Any functions such as clear, the calculate, and the positive and negative methods all needed to be part of the state. This was because these were the functions that handled calling the methods for the actual parsing and calculations and updates the display. This app can't be implemented using a stateless widget because it requires a display to be updated after almost every press of every button.

Describe your approach to designing the calculator's user interface. What considerations did you make for button sizing, spacing, and layout? How did you ensure your app would work on different screen sizes? Discuss any challenges you faced in making the UI intuitive and user-friendly, and how you addressed them.

When making the user interface, I mainly used the example as a crutch and worked from there. For button sizing and spacing and layout, I wanted it to line up with the display so I tried to make

everything revolve around the display. I made the display a certain size that I figured would allow the buttons to be big enough to read the text in them. Then I made the spacing even and made the text big enough to fit in the boxes properly while not being cut off. I used textoverflow for most of my stuff too. I basically made the ui and changed the devices on the emulator to ensure it would fit. Technical wise, I have no idea. The biggest challenge I faced was figuring out how to change individual boxes colors but I figured it out eventually.

What edge cases did you identify and handle in your calculator app? (Consider scenarios like division by zero, multiple operators in sequence, pressing equals without entering a complete expression, etc.) How did you implement error handling? Provide specific examples of potential errors and your solutions.

1. Starting input and negative numbers

Edge Case: User starts by entering - to input a negative number.

Handled in _operator():

```
if (_input.isEmpty) {
  if (operator == "-") {
    _input = "-";
    _display = _input;
  }
  return;
}
```

Other operators at start (+, *, /) are ignored.

Result: Safe for user to input negative numbers at the beginning.

2. Double operators (e.g., 5++3)

Edge Case: User presses multiple operators consecutively.

Handled in _operator():

```
if (_isOperator(_input[_input.length - 1])) {
  _input = _input.substring(0, _input.length - 1) + operator;
}
```

Replaces the last operator instead of stacking them.

Result: Prevents invalid expressions like 5++3 or 5**2.

3. Changing sign in the middle of input (+/-)

Edge Case: User wants to flip the sign of the current number in an expression.

Handled in _posNeg():

Finds the last operator index:

```
int lastOperatorIndex = _input.lastIndexOf(RegExp(r'[+\-*/]'));
```

Flips sign of the current number after the last operator:

```
if (currentNum.startsWith('-')){
  currentNum = currentNum.substring(1);
} else {
  currentNum = '-$currentNum';
}
```

Works even if:

Expression is empty → does nothing.

Expression has multiple operators → flips the last number.

4. Division by zero

Edge Case: User divides by zero.

Handled in divide():

```
if (num2 == 0){
  return 0;
}
```

Prevents crashes; returns 0.

5. Operator precedence

Edge Case: Expressions like 2+3*4.

Handled in parse() and _hasPrecedence():

```
while (operatorStack.isNotEmpty &&
      _hasPrecedence(operatorStack.last, expression[i])){
  ...
}
```

Multiplication/division evaluated before addition/subtraction.

Result: Correct math order, e.g., 2+3*4 = 14.

6. Negative numbers in expressions

Edge Case: Expressions like -5*3.

Handled in parse():

```
if (expression[i] == '-' && (i == 0 || _isOperator(expression[i - 1]))){
    number += '-';
    i++;
}
```

Correctly identifies unary negative operator at start or after another operator.

Result: -5*3 evaluates to -15 correctly.

7. Input overflow / long numbers

Edge Case: User types an extremely long number.

Partial Handling: _buildDisplay() uses:

overflow: TextOverflow.ellipsis

Display won't crash; long numbers are truncated visually.

Edge Case Not Fully Handled: double.parse() may throw if number is too big. Currently, this triggers the catch in _calculate() → shows Error.

8. Empty input / pressing = first

Edge Case: Press = with no numbers entered.

Handled in _calculate():

if (_input.isEmpty) return;

Does nothing, preventing crashes.

9. Clearing input (AC)

Handled in _allClear(): resets _input, _display, _result, and both stacks.

Result: Safe for repeated AC presses; no errors.

10. Input with multiple negative signs or operators

Edge Case: --5, +-3

_operator() replaces last operator → prevents stacking.

_posNeg() correctly toggles last number sign.

How did you organize your code to make it maintainable and readable? Discuss your use of functions, variable naming conventions, and comments. If you were to refactor this application, what improvements would you make? What Dart or Flutter best practices did you follow, and which ones did you find most challenging to implement?

1. Code Organization and Maintainability

a) Separation of Concerns

You organized your code using a single Stateful widget (_MyHomePageState) that handles both the UI and the logic.

Within the state class, you separated functionality into clear methods:

_allClear() → reset calculator

_posNeg() → flip sign

_calculate() → parse and evaluate expression

_operator() → handle operator input

_handleNum() → handle number input

parse(), _isOperator(), _hasPrecedence(), _applyOperator() → parsing & calculation

✅ This modularization makes it readable and maintainable, because each method has a clear, single responsibility.

b) Functions

Functions are small and focused, e.g., add(), subtract(), multiply(), divide().

_buildDisplay() and _buildButtonsGrid() handle UI components separately from the logic.

_buildButton() is reusable, with parameters for text, color, and flex, reducing repeated code.

✅ This avoids duplication and improves readability.

c) Variable Naming

Names are descriptive and consistent:

_display, _input → clearly represent what user sees vs. internal input

numberStack, operatorStack → stack-based parsing is clear

_operand, _oper → button color roles

parse(), _applyOperator(), _hasPrecedence() → self-explanatory

Private members are prefixed with _ as per Dart convention. ✅

💡 Minor improvement: _oper and _operand could be renamed more clearly, e.g., _numberButtonColor and _operatorButtonColor.

d) Comments

You included comments explaining:

Edge cases in _posNeg()

Purpose of methods like _applyOperator()

Most methods are self-explanatory, so comments are minimal but targeted. ✅

💡 Suggestion: Add doc comments for public methods and widget builders for maintainability:

```
/// Builds the calculator display widget.
Widget _buildDisplay() { ... }
```

e) UI Structure

Used Column for vertical layout and GridView.count for buttons.

shrinkWrap: true avoids infinite height issues.

Consistent spacing and padding make it readable visually.

2. Best Practices Followed

Use of setState() correctly to update UI after state changes.

Private methods and variables prefixed with _.

Constants for colors (_operand, _oper) for easy theming.

Reusable widgets (_buildButton()) reduce redundancy.

Error handling in _calculate() using try/catch.

Operator precedence and stack-based parsing, following proper algorithmic principles.

Grid-based responsive layout using GridView.count and flexible buttons.

3. Challenges in Implementing Best Practices

Separating UI and Logic – Currently everything is in _MyHomePageState, making testing harder.

Best practice: Use MVC / MVVM or a state management solution (like Provider or Riverpod).

Decimal and percent handling – Parsing and input validation is tricky with user input.

Button flexibility – Handling 0 with flex: 2 in GridView required careful layout planning.

Edge cases in expressions – Parsing negative numbers, multiple operators, and operator precedence took careful design.

4. Refactoring / Improvements

If I were to refactor:

a) Separate UI and Logic

Create a Calculator model class:

```
class Calculator {
  String input = '';
  double? result;

  void appendNumber(String num) { ... }
  void appendOperator(String op) { ... }
  void calculate() { ... }
  void toggleSign() { ... }
  void clear() { ... }
```

}

Then, _MyHomePageState would only handle UI and call Calculator methods. ✅ Improves testability.

b) Input Validation

Prevent multiple decimals in a number.

Handle percent (%) button.

Show "Error" for division by zero instead of returning 0.

c) UI Improvements

Use Expanded instead of fixed widths for better responsiveness.

Use themes for consistent button and text styles.

Replace GridView with Table for more predictable button layouts.

d) State Management

For a larger app, consider Provider, Riverpod, or Bloc to decouple state from UI.

e) Documentation

Use /// doc comments for all public methods.

Add a README explaining the parsing algorithm and button mapping.

5. Summary

Strengths

Clear, modular functions

Descriptive variable naming

Correct edge case handling for negative numbers, double operators, operator precedence

Reusable UI widgets

Potential Improvements

Separate UI and logic

Add decimal validation and percent handling

Use responsive design for buttons

Consider proper state management for scalability

Improve comments and documentation